

Algorithm

Main Program

1. Start.
2. Display the menu:
 - Matrix addition — prove commutative property.
 - Matrix multiplication — prove non-commutative property.
 - Exit.
3. Read the user's choice.
4. Use **switch** to call the corresponding function.
5. Repeat until the user selects Exit.
6. Stop.

Operation 1: Matrix Addition

1. Input positive values for rows and columns.
2. Declare two VLAs A and B of the same dimensions.
3. Input both matrices.
4. Calculate $A + B$.
5. Calculate $B + A$.
6. Display both results.
7. Compare corresponding elements.
8. Since they are equal, demonstrate that:
 $A + B = B + A$.
9. Therefore, matrix addition is commutative.

Operation 2: Matrix Multiplication

1. Input the dimensions of matrices A and B.
2. To calculate both AB and BA, require:
 - columns of A = rows of B
 - columns of B = rows of A
3. Declare the two VLAs and input their elements.
4. Calculate AB.
5. Calculate BA.
6. Display both products.
7. If their dimensions differ, they cannot be equal.
8. Otherwise, compare corresponding elements.
9. If $AB \neq BA$, the entered matrices demonstrate that matrix multiplication is not commutative.
10. If $AB = BA$, explain that these particular matrices commute, but multiplication is **not commutative in general**.

Test Case Table

Test Case	Choice	Input	Expected Result
1	1	$A = [[1, 2], [3, 4]], B = [[5, 6], [7, 8]]$	$A+B = B+A = [[6, 8], [10, 12]]$; addition is commutative

2	1	$A = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}, B = \begin{bmatrix} 1 & 3 \\ 5 & 1 \end{bmatrix}$	Both sums are equal
3	2	$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, B = \begin{bmatrix} 3 & 4 \\ 1 & 0 \end{bmatrix}$	$AB \neq BA$; demonstrates non-commutativity
4	2	$A = \begin{bmatrix} 1 & 0 \\ 2 & 3 \end{bmatrix}, B = \begin{bmatrix} 0 & 1 \\ 4 & 5 \end{bmatrix}$	$AB = BA$; these particular matrices commute
5	2	A is 2x3, B is 3x2	AB and BA both exist but have different dimensions, so they cannot be equal
6	1/2	Rows or columns ≤ 0	Error; dimensions requested again
7	2	A = 2x3, B = 2x3	Error because both AB and BA cannot be
8	3	—	Program exits
9	5	—	Invalid choice message

C Program

```
#include <stdio.h>
```

```
void inputMatrix(int r, int c, int a[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}
```

```
void displayMatrix(int r, int c, int a[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%d\t", a[i][j]);
        }
        printf("\n");
    }
}
```

```

void matrixAddition(void)
{
    int r, c, i, j, equal = 1;

    do
    {
        printf("Enter rows and columns: ");
        scanf("%d %d", &r, &c);

        if (r <= 0 || c <= 0)
            printf("Rows and columns must be positive.\n");

    } while (r <= 0 || c <= 0);

    int A[r][c], B[r][c];
    int AB[r][c], BA[r][c];

    printf("Enter elements of matrix A:\n");
    inputMatrix(r, c, A);

    printf("Enter elements of matrix B:\n");
    inputMatrix(r, c, B);

    /* Calculate A + B and B + A */
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            AB[i][j] = A[i][j] + B[i][j];
            BA[i][j] = B[i][j] + A[i][j];
        }
    }

    printf("\nA + B:\n");
    displayMatrix(r, c, AB);

    printf("\nB + A:\n");
    displayMatrix(r, c, BA);

    /* Compare A+B and B+A */
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {

```

```

        if (AB[i][j] != BA[i][j])
            equal = 0;
    }
}

if (equal)
    printf("\nA + B = B + A\n"
           "Hence, matrix addition is commutative.\n");
}

void matrixMultiplication(void)
{
    int r1, c1, r2, c2;
    int i, j, k;
    int equal = 1;

    do
    {
        printf("Enter rows and columns of matrix A: ");
        scanf("%d %d", &r1, &c1);

        printf("Enter rows and columns of matrix B: ");
        scanf("%d %d", &r2, &c2);

        if (r1 <= 0 || c1 <= 0 || r2 <= 0 || c2 <= 0)
        {
            printf("Dimensions must be positive.\n");
        }
        else if (c1 != r2 || c2 != r1)
        {
            printf("Both AB and BA must be possible.\n");
            printf("Required: c1 = r2 and c2 = r1.\n");
        }

    } while (r1 <= 0 || c1 <= 0 ||
             r2 <= 0 || c2 <= 0 ||
             c1 != r2 || c2 != r1);

    int A[r1][c1], B[r2][c2];
    int AB[r1][c2], BA[r2][c1];

    printf("Enter elements of matrix A:\n");
    inputMatrix(r1, c1, A);

    printf("Enter elements of matrix B:\n");

```

```

inputMatrix(r2, c2, B);

/* Calculate AB */
for (i = 0; i < r1; i++)
{
    for (j = 0; j < c2; j++)
    {
        AB[i][j] = 0;

        for (k = 0; k < c1; k++)
        {
            AB[i][j] += A[i][k] * B[k][j];
        }
    }
}

/* Calculate BA */
for (i = 0; i < r2; i++)
{
    for (j = 0; j < c1; j++)
    {
        BA[i][j] = 0;

        for (k = 0; k < c2; k++)
        {
            BA[i][j] += B[i][k] * A[k][j];
        }
    }
}

printf("\nA x B:\n");
displayMatrix(r1, c2, AB);

printf("\nB x A:\n");
displayMatrix(r2, c1, BA);

/*
    If dimensions are different, AB and BA
    cannot be equal matrices.
*/
if (r1 != r2 || c2 != c1)
{
    equal = 0;
}
else

```

```

    {
        for (i = 0; i < r1; i++)
        {
            for (j = 0; j < c2; j++)
            {
                if (AB[i][j] != BA[i][j])
                    equal = 0;
            }
        }
    }

    if (!equal)
    {
        printf("\nAB != BA\n");
        printf("Hence, this example demonstrates that "
            "matrix multiplication is not commutative.
\n");
    }
    else
    {
        printf("\nAB = BA for these particular matrices.\n");
        printf("These matrices commute, but matrix
multiplication "
            "is not commutative in general.\n");
    }
}

int main(void)
{
    int choice;

    do
    {
        printf("\n===== MATRIX OPERATIONS
=====\\n");
        printf("1. Prove matrix addition is commutative\\n");
        printf("2. Demonstrate matrix multiplication is not
commutative\\n");
        printf("3. Exit\\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:

```

```
        matrixAddition();
        break;

    case 2:
        matrixMultiplication();
        break;

    case 3:
        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice. Enter 1, 2, or
3.\n");
    }

    } while (choice != 3);

    return 0;
}
```

Execution

```

Last login: Fri Sep 25 19:14:18 on ttys001
[bhavya@Bhavyas-MacBook-Air ~ % nano day32.c
[bhavya@Bhavyas-MacBook-Air ~ % clang day32.c -o day32
[bhavya@Bhavyas-MacBook-Air ~ % ./day32

===== MATRIX OPERATIONS =====
1. Prove matrix addition is commutative
2. Demonstrate matrix multiplication is not commutative
3. Exit
Enter your choice: 1
Enter rows and columns: 2 2
Enter elements of matrix A:
1 2
3 4
Enter elements of matrix B:
5 6
7 8

A + B:
6      8
10     12

B + A:
6      8
10     12

A + B = B + A
Hence, matrix addition is commutative.

===== MATRIX OPERATIONS =====
1. Prove matrix addition is commutative
2. Demonstrate matrix multiplication is not commutative
3. Exit
Enter your choice: █

```

Thus, **matrix addition is commutative** because $A + B = B + A$ for matrices of the same order, whereas **matrix multiplication is not commutative in general** because a counterexample can have $AB \neq BA$.